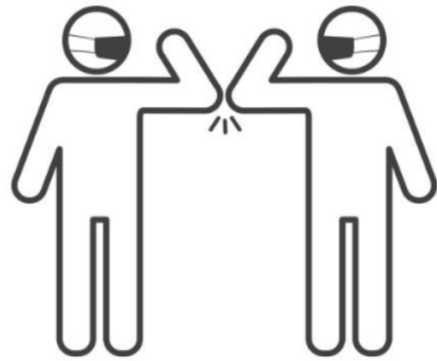




1. Die Elbow-Methode im Clustering

2. Die Elbow-Methode bei der PCA

3. Data Science Projektmanagement

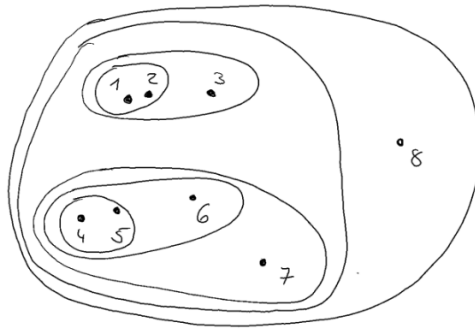
**Abschließend: Zusammenfassung
und Fragen**

```
In [3]: ▶ import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

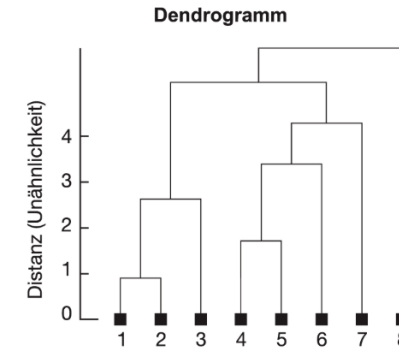
# hack to avoid kmeans warning:
#   UserWarning: KMeans is known to have a memory leak on Windows with MKL,
#   when there are less chunks than available threads.
#   You can avoid it by setting the environment variable OMP_NUM_THREADS=1.
import os
os.environ["OMP_NUM_THREADS"] = "1"
```

Wiederholung: Hierarchisches Clustering

Skizze:



Dendrogramm:



<https://wirtschaftslexikon.gabler.de/definition/dendrogramm-31008>

Frage: Wie funktioniert hierarchisches (agglomeratives) Clustering?

Man sucht zuerst die beiden Punkte mit dem geringsten Abstand und verschmilzt diese zu einem Cluster. Dann iteriert man; hierfür muss eine **Clustermetrik** ("Linkage method") definiert werden, um Abstände zwischen Punkten und Cluster (oder zwischen Cluster und Cluster) zu berechnen.

Beispiel: Für die *Linkage*-Methode `method="single"` wird in der Clustermetrik die geringste Punktmatrix zwischen Punkten aus den beiden Clustern verwendet:

$$d_{\text{single}}(C_1, C_2) = \min_{x_1 \in C_1} \min_{x_2 \in C_2} d_{\text{Punkt}}(x_1, x_2)$$

Einzelne Punkte werden als ein-elementiges Cluster behandelt: $C = \{x\}$.

Wiederholung: Linkagemethoden, um aus einer Punktmetrik eine Clustermetrik abzuleiten:

- `method="single"` : Die kleinste Punktmetrik zwischen Punkten aus den beiden Clustern:

$$d_{\text{single}}(C_1, C_2) = \min_{x_1 \in C_1} \min_{x_2 \in C_2} d_{\text{Punkt}}(x_1, x_2)$$

- `method="complete"` oder `"maximum"` : Die größte Punktmetrik:

$$d_{\text{complete}}(C_1, C_2) = \max_{x_1 \in C_1} \max_{x_2 \in C_2} d_{\text{Punkt}}(x_1, x_2)$$

- `method="average"` : Der Durchschnitt der Punktmetriken zwischen Punkten aus den beiden Clustern:

$$d_{\text{average}}(C_1, C_2) = \frac{1}{C_1 \cdot C_2} \sum_{x_1 \in C_1} \sum_{x_2 \in C_2} d_{\text{Punkt}}(x_1, x_2)$$

In der Linkage-Methode `method="ward"` wird keine Clustermetrik betrachtet:

- `method="ward"` : Man beginnt mit einelementigen Clustern. Für jeden potentiellen nächsten Schritt ($C' \rightarrow C' \cup C''$) wird folgende Abweichung von den Clustermittelpunkten ("Varianz") berechnet:

$$V = \sum_C \sum_{x \in C} d_{\text{euclid}}(x, \bar{x}_C)^2, \text{ wobei } (\bar{x}_C)_i = \frac{1}{|C|} \sum_{x \in C} x_i$$

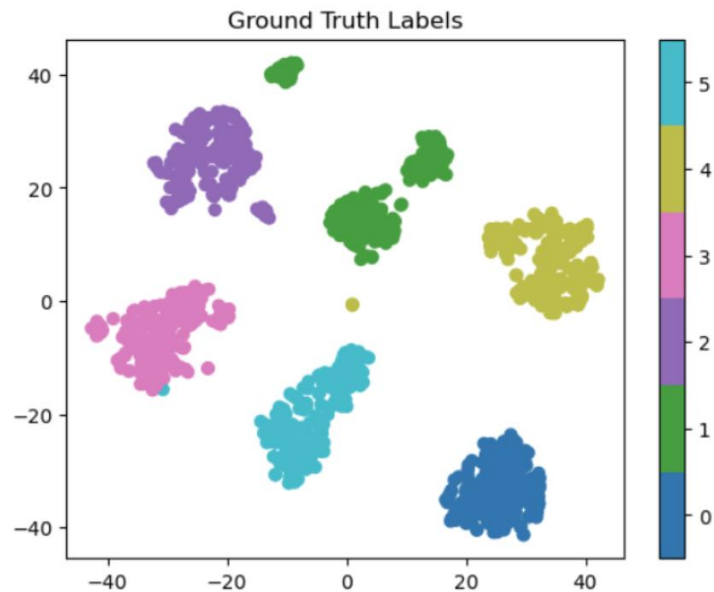
Der nächste Schritt mit der niedrigsten "Varianz" wird dann ausgeführt.

Wiederholung: Beispiel digits -Datensatz

- hierbei wurden zum Visualisieren die 8x8 Pixel mittels t-SNE auf 2 Dimensionen projiziert (Ergebnis: `data_trf`)
- die ursprünglichen 64-Dim-Daten geclustert (mittels `AgglomerativeClustering(n_clusters=10)`)

Slide Type

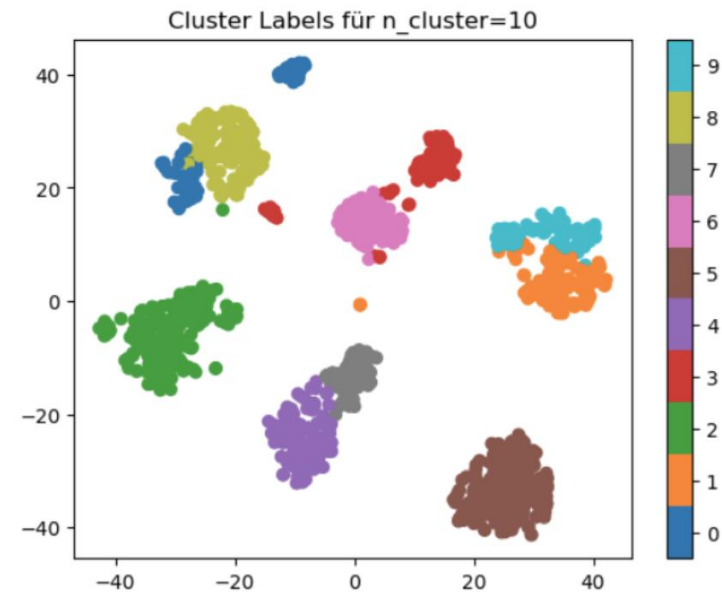
```
plt.scatter (data_trf[:,0], data_trf[:,1],  
            c=target,  
            cmap=plt.get_cmap("tab10",6),  
            vmin=-0.5, vmax=5.5 )  
cbar = plt.colorbar(ticks=np.arange(0,7,1) )  
plt.title("Ground Truth Labels")  
plt.show()
```



In [158]:

Slide Type

```
plt.scatter (data_trf[:,0], data_trf[:,1],  
            c=model.labels_,  
            cmap=plt.get_cmap("tab10",10),  
            vmin=-0.5, vmax=9.5 )  
cbar = plt.colorbar(ticks=np.arange(0,10,1) )  
plt.title("Cluster Labels für n_cluster=10")  
plt.show()
```



Lade den `digits` Datensatz wie zuvor. Nun Beschränkung auf 50 Bilder und nur 4 Klassen.

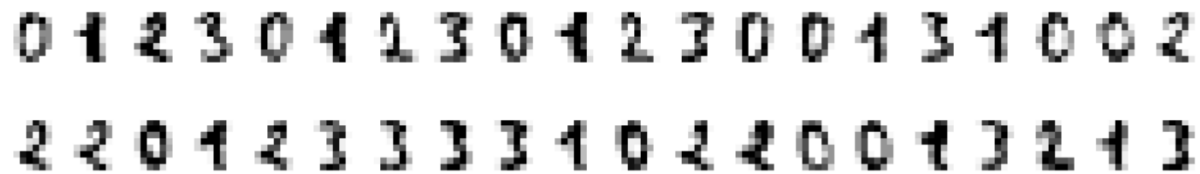
```
In [4]: ▶ from sklearn.datasets import load_digits
digits = load_digits( n_class=4 )
target = digits["target"][:50]
data = digits["data"][:50]
target.shape, data.shape
```

Out[4]: ((50,), (50, 64))

```
In [5]: ▶ pd.Series(target).value_counts()
```

```
Out[5]: 1    13
        3    13
        0    12
        2    12
        Name: count, dtype: int64
```

```
In [6]: ▶ fig, axes = plt.subplots(nrows=2, ncols=20, figsize=(12, 2))
for ax, image in zip(axes.flatten(), data[:40]):
    ax.imshow( image.reshape(8, 8), cmap="Greys" )
    ax.set_axis_off()
```



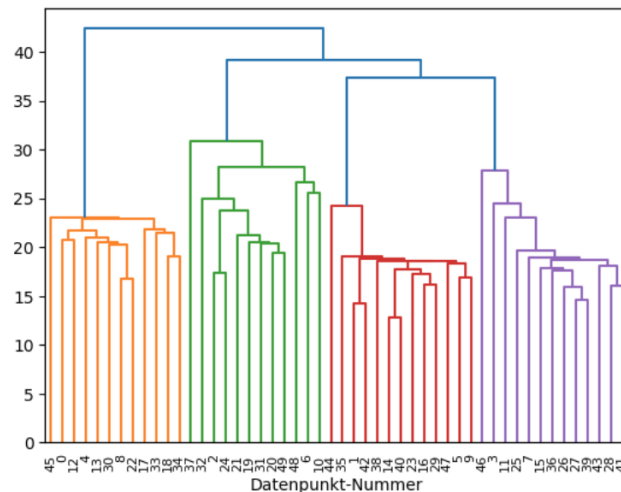
Statt der `scikit-learn` Funktionalität können wir die `scipy` Funktionalität verwenden, die (für Hierarchisches Clustering) etwas bequemer ist, da sie eine nachträgliche Wahl von `n_clusters` zulässt.

```
In [243]: >> from scipy.cluster.hierarchy import linkage  
          from scipy.spatial.distance import pdist
```

```
In [265]: >> Z = linkage(pdist(data), method="single")  
          Z.shape  
          # Z beinhaltet das komplette Dendrogramm. Mit Anwendung von fcluster(Z,t=33) kann dieses Dendrogramm  
          # bei z.B. Höhe 33 "abgeschnitten" werden und man erhält dann die entsprechenden Cluster.
```

Out[265]: (49, 4)

```
In [283]: >> from scipy.cluster.hierarchy import dendrogram  
          dendrogram ( Z, color_threshold=33 )  
          plt.xlabel("Datenpunkt-Nummer");
```



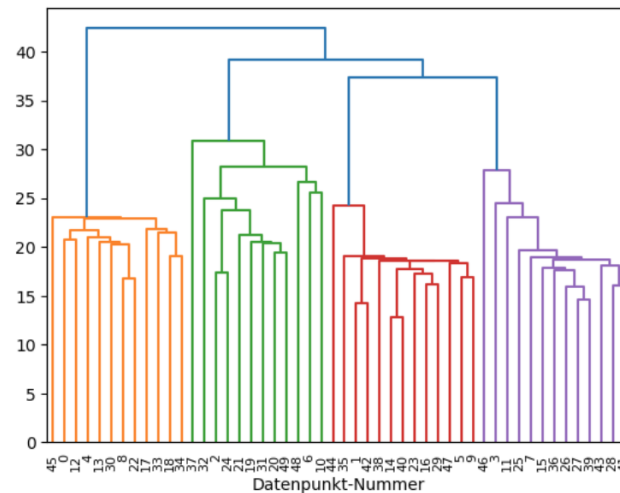
Hier wurde der `color_threshold=33` gesetzt, um vier Cluster zu bekommen.

Frage: Ist das ein "guter" Threshold?

Wir können (in einem "überwachten"/supervised Szenario) mit der Ground-Truth vergleichen.

```
In [284]: ▶ from scipy.cluster.hierarchy import dendrogram
dendrogram ( Z, color_threshold=33 );
xticklabels = plt.gca().get_xticklabels ()
plt.xlabel("Datenpunkt-Nummer");
xticklabels[:5]
```

```
Out[284]: [Text(5, 0, '45'),
Text(15, 0, '0'),
Text(25, 0, '12'),
Text(35, 0, '4'),
Text(45, 0, '13')]
```



```
In [269]: ▶ # die Ground-Truth für die ersten 5 Punkte von Links
target [ 45 ] , \
target [ 0 ] , \
target [ 12 ] , \
target [ 4 ] , \
target [ 13 ]
```

```
Out[269]: (0, 0, 0, 0, 0)
```

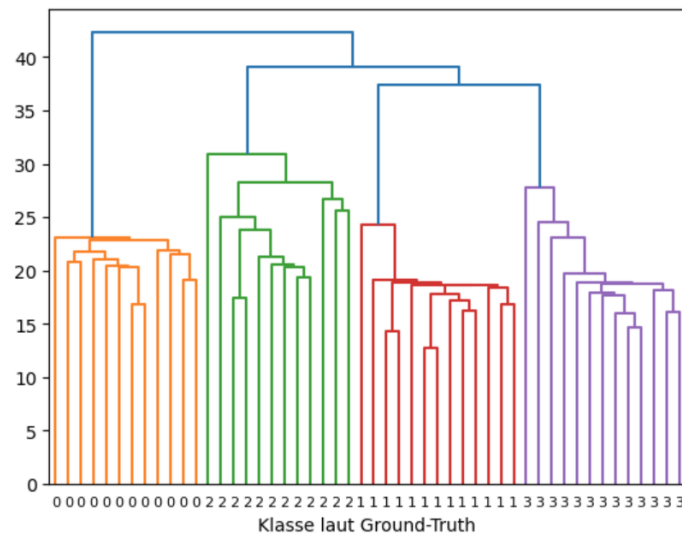
```
In [271]: ▶ # äquivalent, automatisierbar
target [ int(xticklabels[0].get_text()) ] , \
target [ int(xticklabels[2].get_text()) ] , \
target [ int(xticklabels[3].get_text()) ] , \
target [ int(xticklabels[4].get_text()) ] , \
target [ int(xticklabels[5].get_text()) ]
```

```
Out[271]: (0, 0, 0, 0, 0)
```

Das Dendrogramm scheint (ganz links) gut mit der Ground Truth übereinzustimmen.

Systematischer Vergleich mit der Ground Truth:

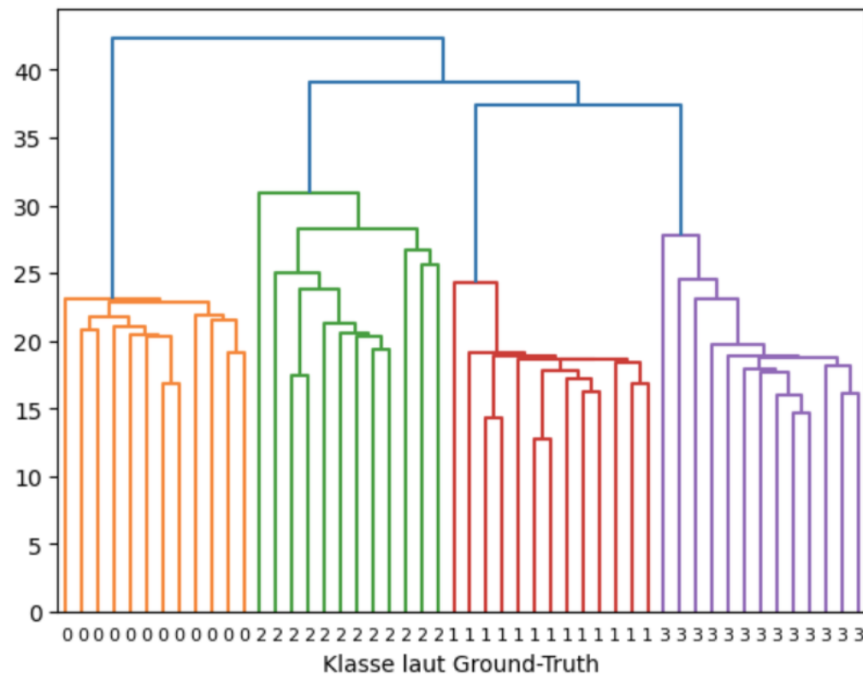
```
In [280]: ▶ Z = linkage(pdist(data), method="single")
from scipy.cluster.hierarchy import dendrogram
dendrogram ( Z, color_threshold=33 );
xticklabels = plt.gca().get_xticklabels ()
plt.gca().set_xticklabels ( [target[int(xticklabels[i].get_text())] for i in range(len(xticklabels))], rotation=0 )
plt.xlabel("Klasse laut Ground-Truth");
```



Für diesen Threshold: Das Dendrogramm stimmt mit der Ground Truth gut überein.

Frage: Wie bestimmt man "unsupervised" einen guten Threshold?

Idee: Berechne die Inertia: Wie weit sind die Punkte vom Mittelpunkt ihres Clusters entfernt?



```
In [290]: # Weise cluster Labels zu für Höhe t=33
# (analog model.Labels_ bei sklearn, aber einstellbar)
from scipy.cluster.hierarchy import fcluster
labels = fcluster(Z,t=33,criterion="distance")
labels
```

```
Out[290]: array([1, 3, 2, 4, 1, 3, 2, 4, 1, 3, 2, 4, 1, 1, 3, 4,
3, 1, 1, 2, 2, 2,
1, 3, 2, 4, 4, 4, 4, 3, 1, 2, 2, 1, 1, 3, 4, 2,
3, 4, 3, 4, 3, 4,
3, 1, 4, 3, 2, 2], dtype=int32)
```

```
In [299]: # Cluster mit dem Bild "0"
idx = np.where(labels==1)[0]
idx
```

```
Out[299]: array([ 0, 4, 8, 12, 13, 17, 18, 22, 30, 33, 34, 45],
dtype=int64)
```

```
In [311]: # SSE: Summe der (quadrierten) Abstände der Punkte
# zu ihrem Cluster-Mittelpunkt (sum of squared errors)
cluster_center = data[idx].mean(axis=0) # 64-dim vector
cluster_SSE = ((data[idx]-cluster_center)**2) \
.sum(axis=1).sum()
cluster_SSE
```

```
Out[311]: 4358.25
```

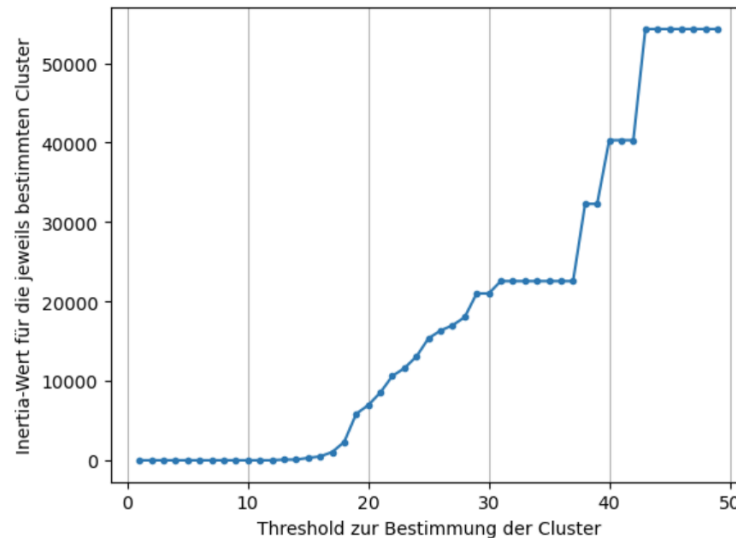
```
In [318]: # Die Inertia ist die Summe der SSEs, summiert über alle Cluster:
def inertia(labels,data):
    SSEs = dict()
    for label in np.unique(labels):
        idx = np.where(labels==label)[0]
        cluster_center = data[idx].mean(axis=0)
        SSEs[label] = ((data[idx]-cluster_center)**2).sum(axis=1).sum()
    return sum( list(SSEs.values()) )
inertia(labels,data)
```

```
Out[318]: 22557.692307692305
```

Vergleiche nun die Inertia für verschiedene Threshold-Werte:

Suche einen "Knick": Von rechts nach links kommend, nimmt die Anzahl der Cluster immer weiter zu (da der Treshold kleiner wird). Ab einem gewissen Punkt "lohnt" es sich nicht mehr, mehr Cluster zu haben.

```
In [332]: ▶ pd.Series ( {t: inertia( fcluster(Z,t,criterion="distance"),data) for t in range(1,50)} ).plot( marker=".");
plt.grid(axis="x")
plt.xlabel("Threshold zur Bestimmung der Cluster")
plt.ylabel("Inertia-Wert für die jeweils bestimmten Cluster");
```



```
In [329]: ▶ np.unique ( fcluster(Z,t=37,criterion="distance") )
```

```
Out[329]: array([1, 2, 3, 4], dtype=int32)
```

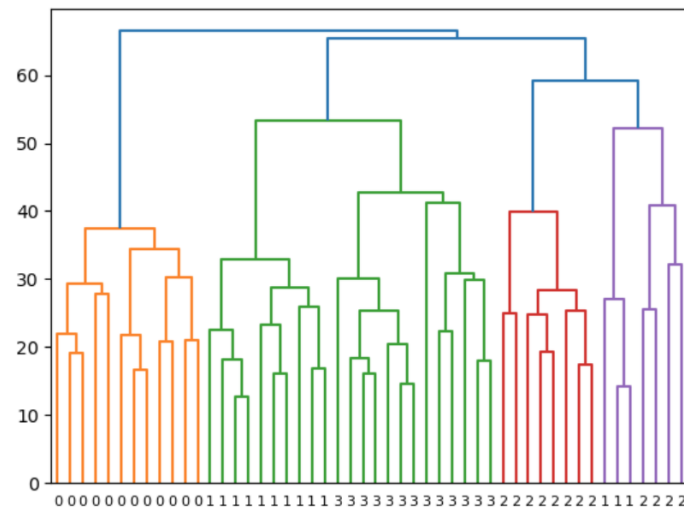
```
In [330]: ▶ np.unique ( fcluster(Z,t=38,criterion="distance") )
```

```
Out[330]: array([1, 2, 3], dtype=int32)
```

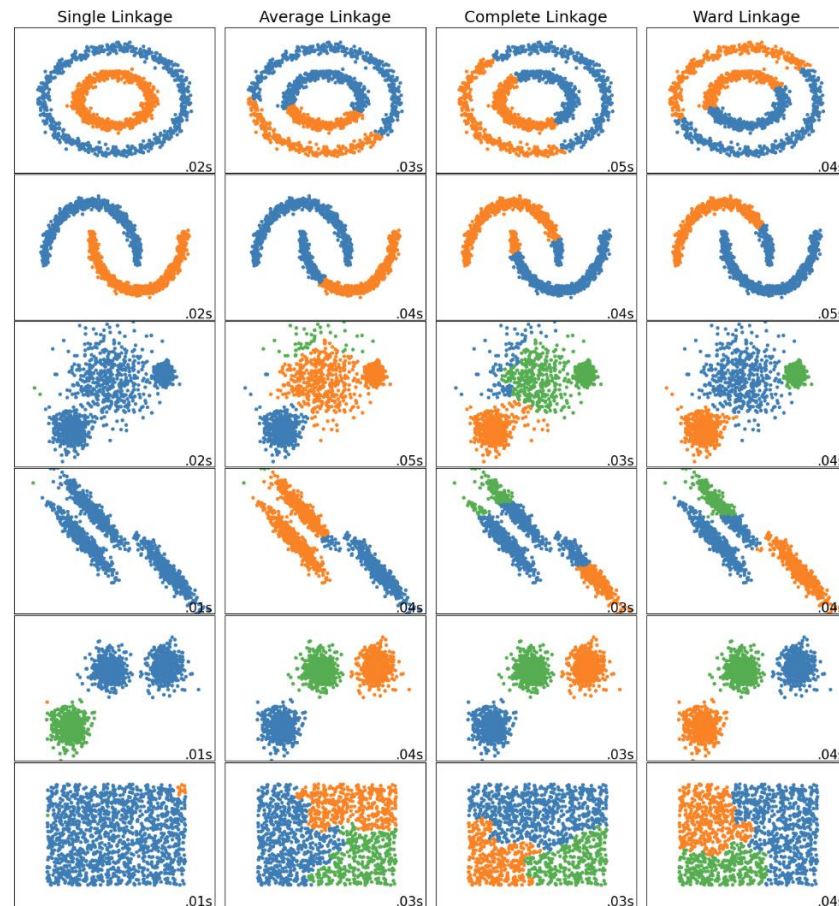
Dieser "Knick" wird auch "Elbow" genannt. Daher heißt diese Methode (auch in anderen Situationen, wenn ein Hyperparameter schrittweise verändert wird und die Auswirkungen bewertet werden), "Elbow-Methode".

Warnung: Im Vergleich dazu method="complete": Kein korrektes Clustering möglich!

```
In [277]: ▶ Z = linkage(pdist(data), method="complete")  
from scipy.cluster.hierarchy import dendrogram  
dendrogram ( Z, color_threshold=54 );  
xticklabels = plt.gca().get_xticklabels ()  
plt.gca().set_xticklabels ( [target[int(xticklabels[i].get_text())] for i in range(len(xticklabels))], rotation=0 );
```



Linkage-Methoden im Überblick: Auszüge aus <https://scikit-learn.org/stable/modules/clustering.html#hierarchical-clustering>



Agglomerative cluster has a “rich get richer” behavior that leads to uneven cluster sizes. In this regard, **single linkage** is the worst strategy, and **Ward** gives the most regular sizes.

However, the affinity (or distance used in clustering) cannot be varied with Ward, thus for non Euclidean metrics, **average linkage** is a good alternative.

Single linkage, while not robust to noisy data, can be computed **very efficiently** and can therefore be useful to provide hierarchical clustering of larger datasets. Single linkage can also perform well on **non-globular data**.

Abschließend: Neben der Linkage-Methode ("Cluster-Metrik basierend auf einer gegebenen Punkt-Metrik") hat die verwendete Punktmatrix (default: Euklidische Metrik) einen entscheidenden Einfluss auf das Clustering-Ergebnis.

Die Elbow-Methode in anderen Clustering-Verfahren

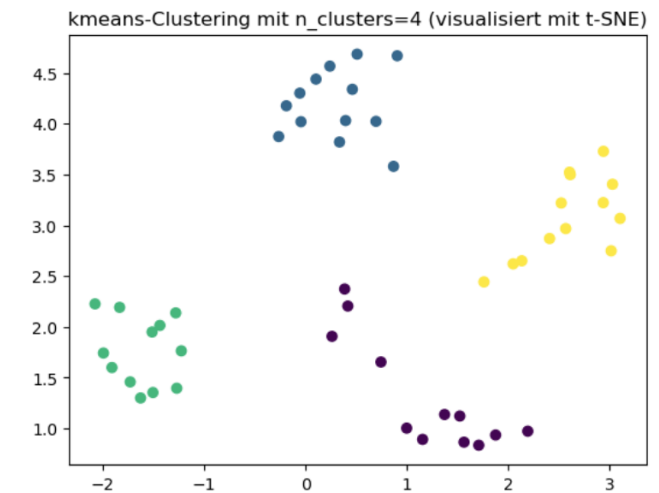
Wir verwenden kmeans (in dieser Vorlesung nicht im Detail erklärt) als einfache Clustering-Methode. Hier muss mit dem Parameter `n_clusters` vorab festgelegt werden, wie viele Cluster gefunden werden sollen.

```
In [22]: ▶ # kmeans Clustering mit 4 Clustern
# (und n_init=10, es wird also das Verfahren
# 10x mit verschiedenen Startbedingungen gestartet
# und das beste Ergebnis verwendet)
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=4, n_init=10)
kmeans.fit ( data )
kmeans.labels_
```

```
Out[22]: array([2, 3, 0, 1, 2, 3, 0, 1, 2, 3, 0, 1, 2, 2, 3, 1,
3, 2, 2, 0, 0, 0, 0,
2, 3, 0, 1, 1, 1, 1, 3, 2, 0, 0, 2, 2, 3, 1, 0,
3, 1, 3, 1, 3, 1,
3, 2, 1, 3, 0, 0])
```

```
In [23]: ▶ from sklearn.manifold import TSNE
data_tsne = TSNE().fit_transform(data)

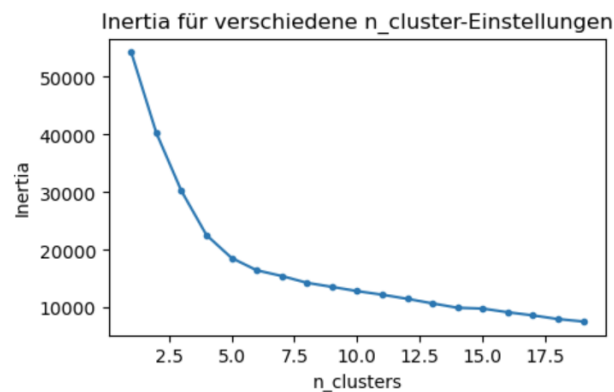
plt.scatter(data_tsne[:,0], data_tsne[:,1],
            c=kmeans.labels_)
plt.title("kmeans-Clustering mit n_clusters=4 "+"
        "(visualisiert mit t-SNE)");
```



Im Beispiel wissen wir, dass `n_clusters=4` optimal ist (weil die Bilder 4 Ziffern zeigen).
Wie können wir `n_clusters` bestimmen, wenn wir kein Vorwissen haben?

```
In [10]: ▶ inertias = dict()
for n in range(1,20):
    kmeans = KMeans(n_clusters=n, n_init=10)
    kmeans.fit ( data )
    inertias[n] = kmeans.inertia_

pd.Series(inertias).plot(marker=".", figsize=(5,3),
    title="Inertia für verschiedene n_cluster-Einstellungen",
    plt.xlabel("n_clusters")
    plt.ylabel("Inertia");
```



Hier werden die Inertia-Werte für verschiedene Cluster-Einstellungen verglichen. Es gibt wieder einen "Elbow", also einen "Knick" bei ca. `n_clusters=4`.

Ab diesem Parameterwert ist der Zusatznutzen weiterer Cluster gering.

Zusammenfassung: Die *Inertia* misst die Entfernung der Punkte zu ihrem Clustermittelpunkt:

$$I = \sum_{\text{Cluster } C} \sum_{x \in C} d_{\text{euclid}}(x, \bar{x}_C)^2,$$

wobei die Komponenten des Clustermittelpunkts definiert sind via

$$(\bar{x}_C)_i = \frac{1}{|C|} \sum_{x \in C} x_i.$$

Beobachtung: Bei `linkage="ward"` wird in jedem Schritt genau die Verschmelzung durchgeführt, die die Inertia minimiert.

Wir überprüfen, ob unsere Inertia-Formel mit der sklearn-Implementierung übereinstimmt:

```
In [19]: ▶ inertias_scik = dict()
          inertias_self = dict()

          for n in range(1,20):
              kmeans = KMeans(n_clusters=n, n_init=10)
              kmeans.fit ( data )
              inertias_scik [n] = kmeans.inertia_
              inertias_self[n] = ((data - kmeans.cluster_centers_[kmeans.labels_])**2).sum(axis=0).sum() # oder einfach .sum()
```

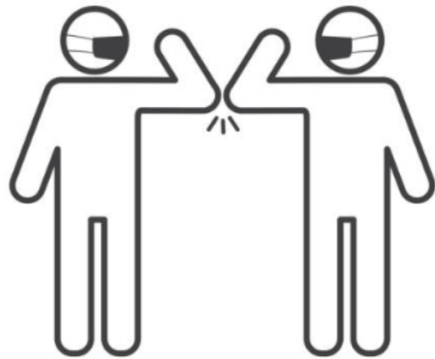
```
In [21]: ▶ inertias_scik
```

```
Out[21]: {1: 54308.98,
          2: 40308.61842105263,
          3: 30226.188375350135,
          4: 22557.692307692305,
          5: 18680.56730769231,
          6: 16396.762179487177,
          7: 15505.012179487177,
          8: 14338.687698412698,
          9: 13641.388888888887,
          10: 12711.983333333334,
          11: 11801.979761904759,
          12: 11659.666666666664,
          13: 10445.722222222223,
          14: 10024.388888888889,
          15: 9710.358333333335,
          16: 9217.183333333332,
          17: 8674.590476190475,
          18: 8086.25,
          19: 7533.516666666665}
```

```
In [20]: ▶ inertias_self
```

```
Out[20]: {1: 54308.979999999996,
          2: 40308.618421052626,
          3: 30226.18837535014,
          4: 22557.692307692305,
          5: 18680.56730769231,
          6: 16396.762179487177,
          7: 15505.012179487181,
          8: 14338.6876984127,
          9: 13641.388888888887,
          10: 12711.983333333334,
          11: 11801.979761904762,
          12: 11659.666666666664,
          13: 10445.722222222223,
          14: 10024.388888888889,
          15: 9710.358333333334,
          16: 9217.183333333334,
          17: 8674.590476190477,
          18: 8086.25,
          19: 7533.516666666666}
```

Ausblick: Mit dem *Silhouette-Index* gibt es zur Clustering-Bewertung eine differenziertere Metrik, die die Nähe eines Punktes nicht nur zum eigenen Clustermittelpunkt bewertet, sondern diesen Abstand zusätzlich in Relation setzt zum Abstand zum nächsten Cluster.



1. Die Elbow-Methode im Clustering

2. Die Elbow-Methode bei der PCA

3. Data Science Projektmanagement

**Abschließend: Zusammenfassung
und Fragen**

2.1 Wiederholung: t-SNE und PCA

```
In [3]: ▶ from sklearn.datasets import load_digits
digits = load_digits( n_class=6 )
target = digits["target"]
data = digits["data"]
target.shape, data.shape
```

```
Out[3]: ((1083,), (1083, 64))
```

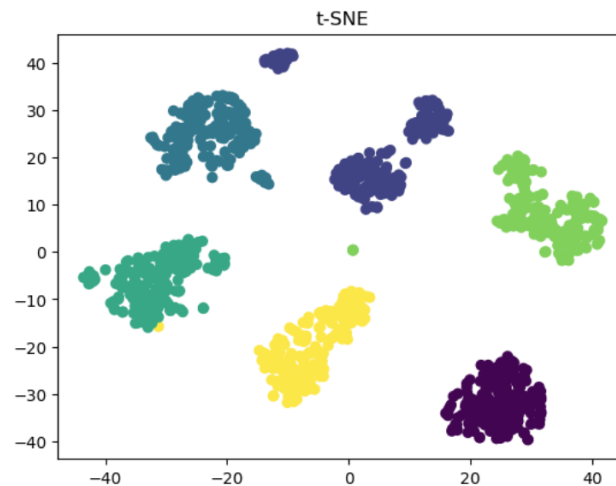
Der sklearn `digits` Datensatz beinhaltet mit der Hand gezeichnete Ziffern:

```
In [9]: ▶ fig, axes = plt.subplots(nrows=2, ncols=20, figsize=(12, 2))
for ax, image in zip(axes.flatten(), data[:40]):
    ax.imshow( image.reshape(8, 8), cmap="Greys" )
    ax.set_axis_off()
```

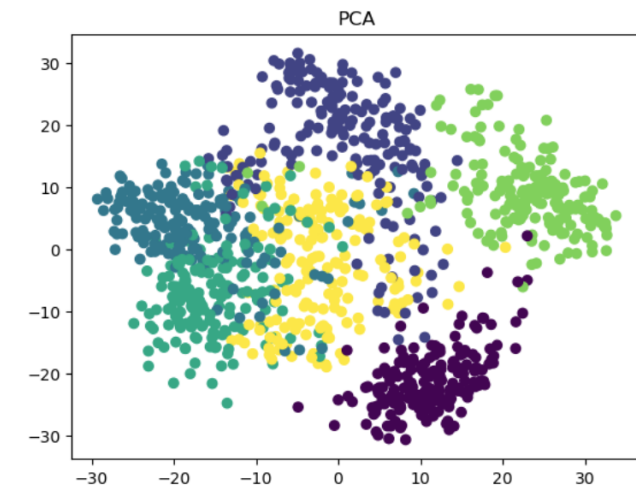


Erinnerung: t-SNE ist eine nicht-lineare Methode, in der versucht wird, mit *Stochastic Gradient Descent* im niedrigdimensionalen Ziel-Raum die Punkt-zu-Punkt-Abstände im hochdimensionalen Ursprungsraum nachzubilden.

```
In [12]: ▶ from sklearn.manifold import TSNE  
data_tsne = TSNE().fit_transform(data)  
plt.scatter(data_tsne[:,0], data_tsne[:,1], c=target)  
plt.title ("t-SNE");
```



```
In [17]: ▶ from sklearn.decomposition import PCA  
data_trf = PCA().fit_transform(data)  
plt.scatter(data_trf[:,0], data_trf[:,1], c=target)  
plt.title ("PCA");
```



Erinnerung: PCA hingegen ist eine lineare Projektionsmethode, in der in jedem Iterationsschritt nach der Richtung der größten Varianz in den Daten gesucht wird und dann in diese Richtung projiziert wird.

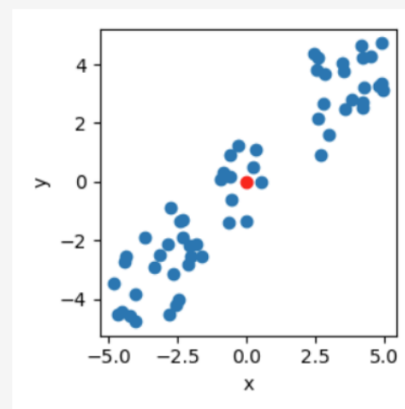
1.2 Die *explained variance ratio* bei einer PCA

In einer PCA wird in jedem Iterationsschritt die Richtung gesucht, in der der eindimensionale Datenvektor, der entsteht, indem man in die Richtung projiziert, die größtmögliche Varianz hat.

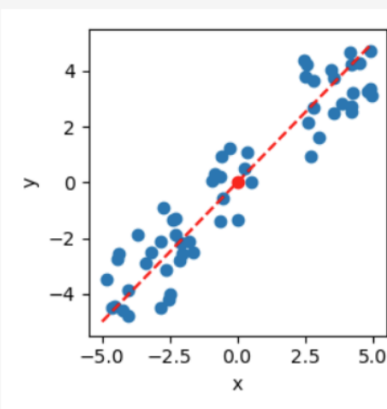
Die in den jeweiligen Schritten bestimmten Varianzen $V_1, V_2, \dots$ sind konstruktionsgemäß absteigend angeordnet:

$$V_1 \geq V_2 \geq V_3 \geq \dots$$

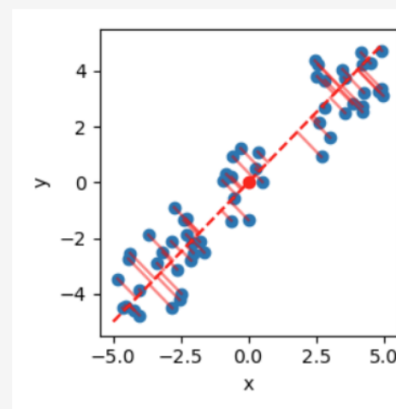
Schritt 1:
Daten zentrieren



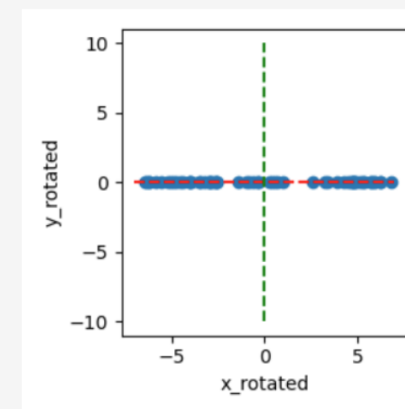
Schritt 2:
Richtung der "max. Varianz" bestimmen



Schritt 3:
Daten projizieren



Schritt 4:
Projizierte Daten als PC1 verwenden

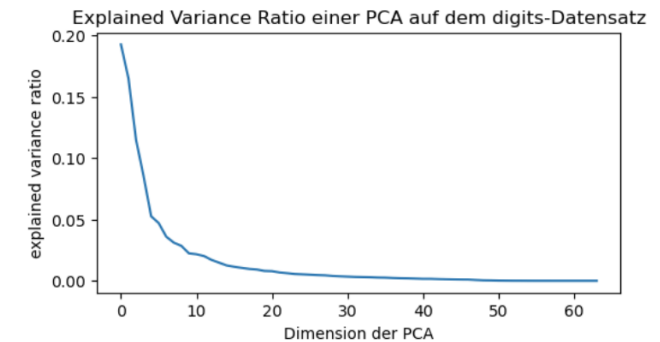


Unter der *explained variance ratio* im *i*-ten Schritt der PCA versteht man die normierte Varianz $V_i / \sum_j V_j$.

```
In [20]: >> pca = PCA()
data_trf = pca.fit_transform(data)
pca.explained_variance_ratio_
```

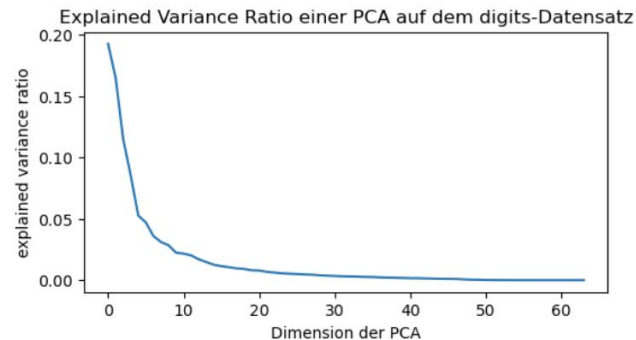
```
Out[20]: array([1.92687518e-01, 1.64914226e-01, 1.14961791e-01,
8.52383819e-02,
5.26076396e-02, 4.70375439e-02, 3.59658880e-02,
3.11125241e-02,
2.84669234e-02, 2.23804557e-02, 2.16326731e-02,
2.01712406e-02,
1.70073119e-02, 1.47557055e-02, 1.25193393e-02,
1.14136476e-02,
1.05064443e-02, 9.65146555e-03, 9.16905085e-03,
8.04020999e-03,
7.83000936e-03, 6.78432338e-03, 6.20556018e-03,
5.54375646e-03,
5.26707974e-03, 4.96200161e-03, 4.66105824e-03,
4.44597812e-03,
3.94136355e-03, 3.65386857e-03, 3.40260435e-03,
3.14310394e-03,
3.01884997e-03, 2.89698311e-03, 2.64977646e-03,
2.55527737e-03,
2.26205957e-03, 2.05829507e-03, 1.95937459e-03,
1.77910328e-03,
1.59412937e-03, 1.58581196e-03, 1.38992998e-03,
1.25153168e-03,
1.13240805e-03, 1.03661235e-03, 9.89760611e-04,
6.64716879e-04,
3.82930069e-04, 3.01148520e-04, 1.65881068e-04,
1.07838064e-04,
5.40506855e-05, 4.90139626e-05, 1.95570435e-05,
5.45578725e-06,
4.21201047e-06, 1.98401289e-06, 1.27176382e-06,
8.28666897e-07,
5.20903775e-07, 1.34378606e-33, 1.34378606e-33,
1.31087728e-33])
```

```
In [30]: >> fig,ax=plt.subplots(figsize=(6,3))
pd.Series(pca.explained_variance_ratio_).plot()
plt.title("Explained Variance Ratio einer PCA " +
"auf dem digits-Datensatz")
plt.xlabel("Dimension der PCA")
plt.ylabel("explained variance ratio");
```



Frage: Welche Dimensionen würden Sie in den transformierten Daten als besonders "informativ" ansehen?

```
In [30]: ▶ fig,ax=plt.subplots(figsize=(6,3))
pd.Series(pca.explained_variance_ratio_).plot()
plt.title("Explained Variance Ratio einer PCA " +
        "auf dem digits-Datensatz")
plt.xlabel("Dimension der PCA")
plt.ylabel("explained variance ratio");
```

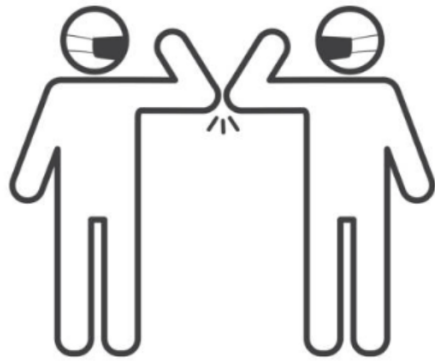


Die ersten Dimensionen, die relativ viel zur Erklärung der Varianz in den Daten beitragen, werden als besonders informativ angesehen.

Die letzten Dimensionen, die nur noch wenig Varianz erklären, werden als "Rauschen" angesehen.

Anwendung der Elbow Method bei der PCA:

Oft sieht man einen "Knick" in der explained variance ratio Kurve. Dieser sogenannte Ellenbogen (engl. *elbow*) wird gerne verwendet, um die Anzahl der informativen Dimensionen zu bestimmen, die als Ergebnis einer PCA festgehalten werden (etwa `n_components=8`).



1. Die Elbow-Methode im Clustering
2. Die Elbow-Methode bei der PCA
3. Data Science Projektmanagement

**Abschließend: Zusammenfassung
und Fragen**



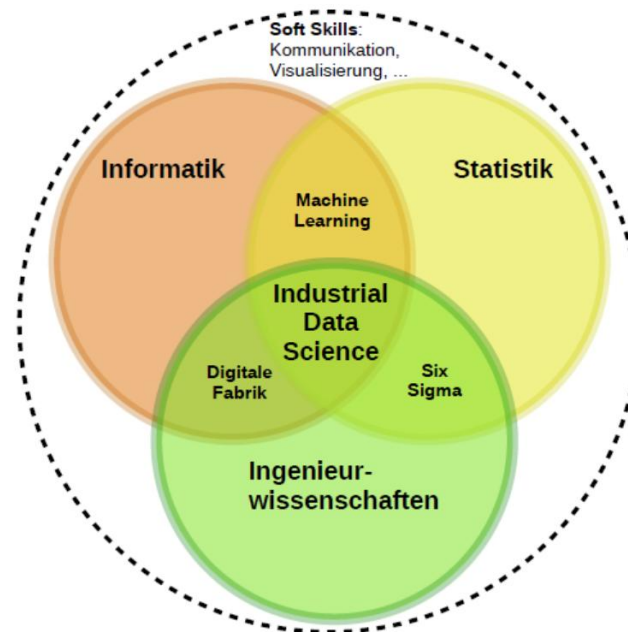
Among leading AI teams, many can likely replicate others' software in, at most, 1-2 years. But it is exceedingly difficult to get access to someone else's data.

Thus data, rather than software, is the defensible barrier for many businesses.

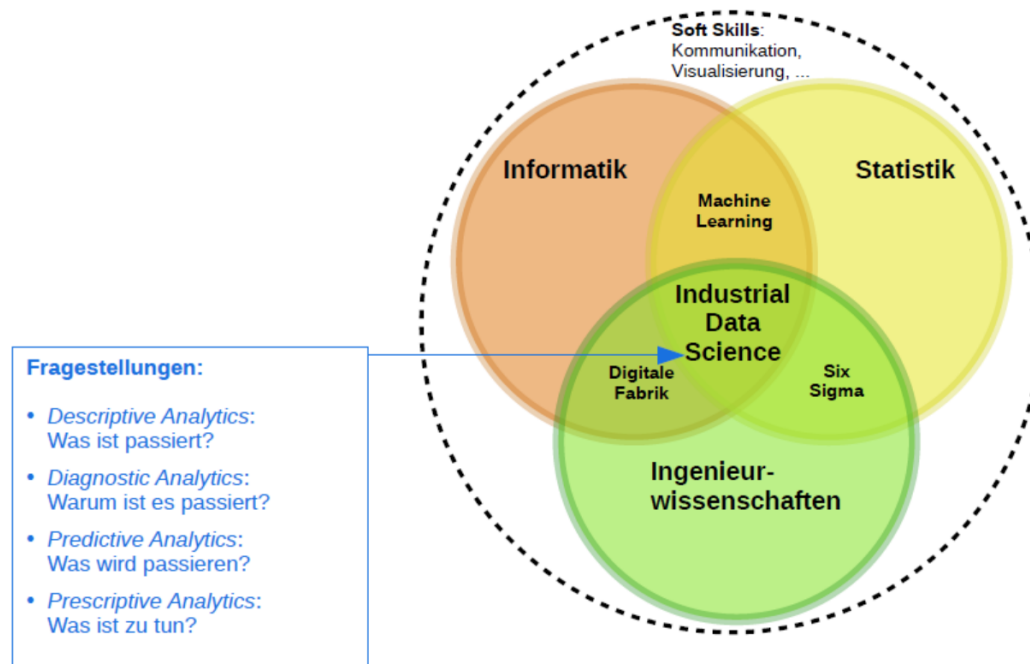
-- Andrew Ng

(u.a. Cofounder Google Brain, Chief Scientist bei Baidu, Director Stanford AI Lab, Cofounder Coursera, Cofounder DeepLearning.AI)

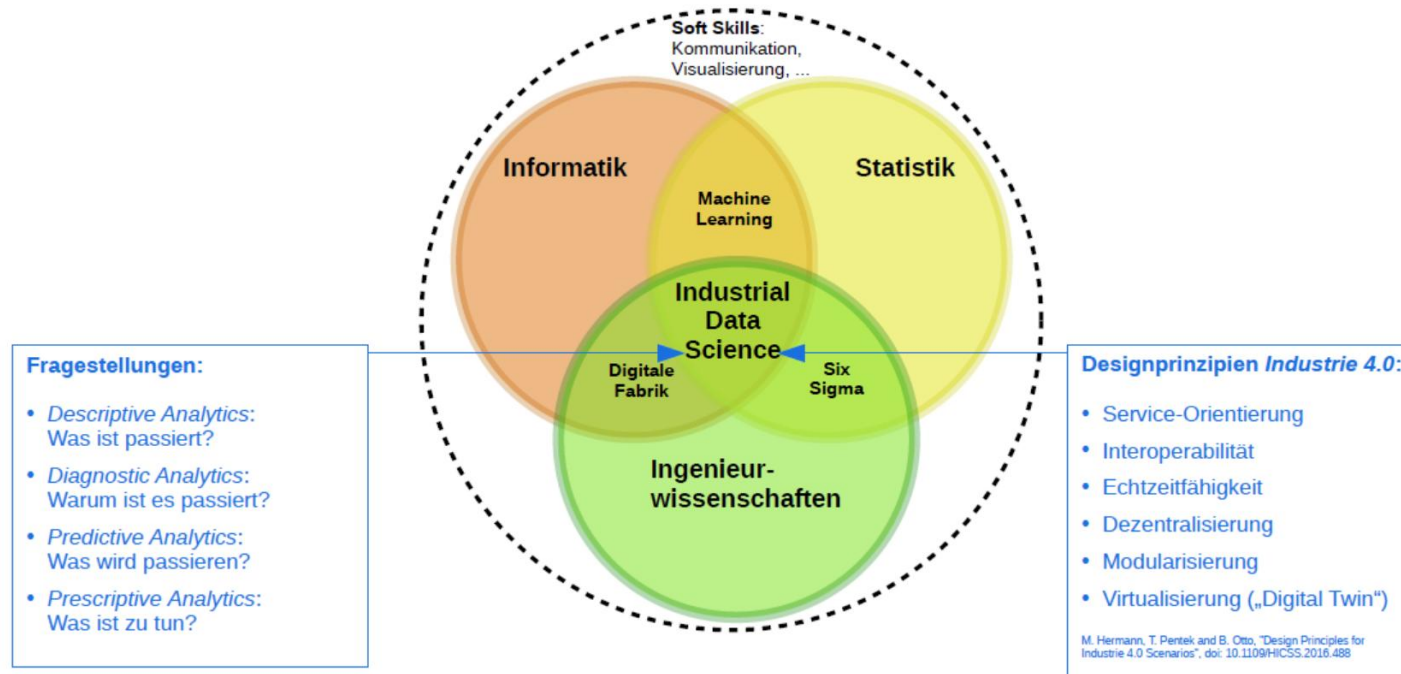
3.1 Beispiel *Industrial Data Science*



Wiederholung: Das Data Science Venn-Diagramm



Wiederholung: Das Gartner Ascendancy Modell



The ability of objects, machines and people to communicate, exchange data and coordinate activities

The ability to create virtual copies of everything to see how new equipment or processes will impact operations

Business logic is contained in sub-systems or components rather than a central computer system, enabling autonomous decisions

Design Principles Industry 4.0



The collection and analysis of data in real-time, allowing decisions to be made immediately and at every moment

Free information flow within and between businesses to better meet customer needs

The ability to flexibly adapt to changing requirements and industry needs

Nach: <https://www.rmit.edu.au/news/c4de/industry-4-0-design-principles>
Basierend auf: M. Hermann, T. Pentek and B. Otto, "Design Principles for Industrie 4.0 Scenarios," 2016 49th Hawaii International Conference on System Sciences (HICSS), Koloa, HI, 2016, pp. 3928-3937, doi: 10.1109/HICSS.2016.488

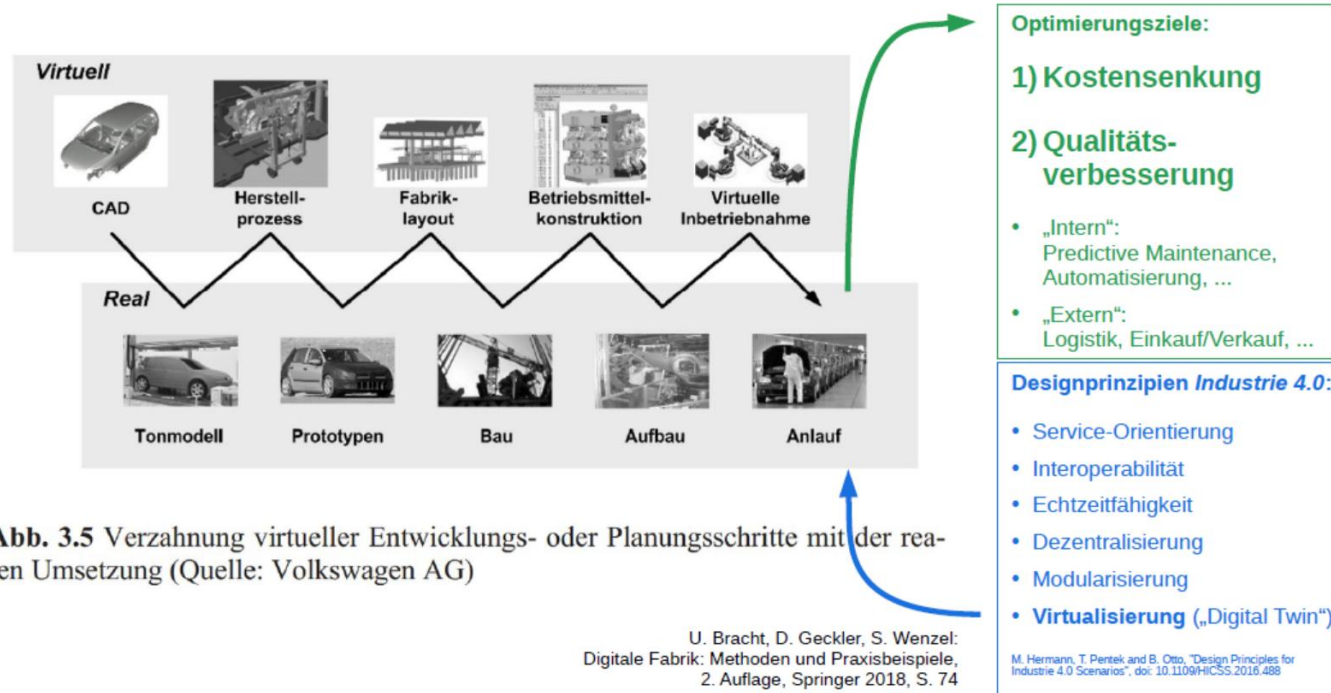
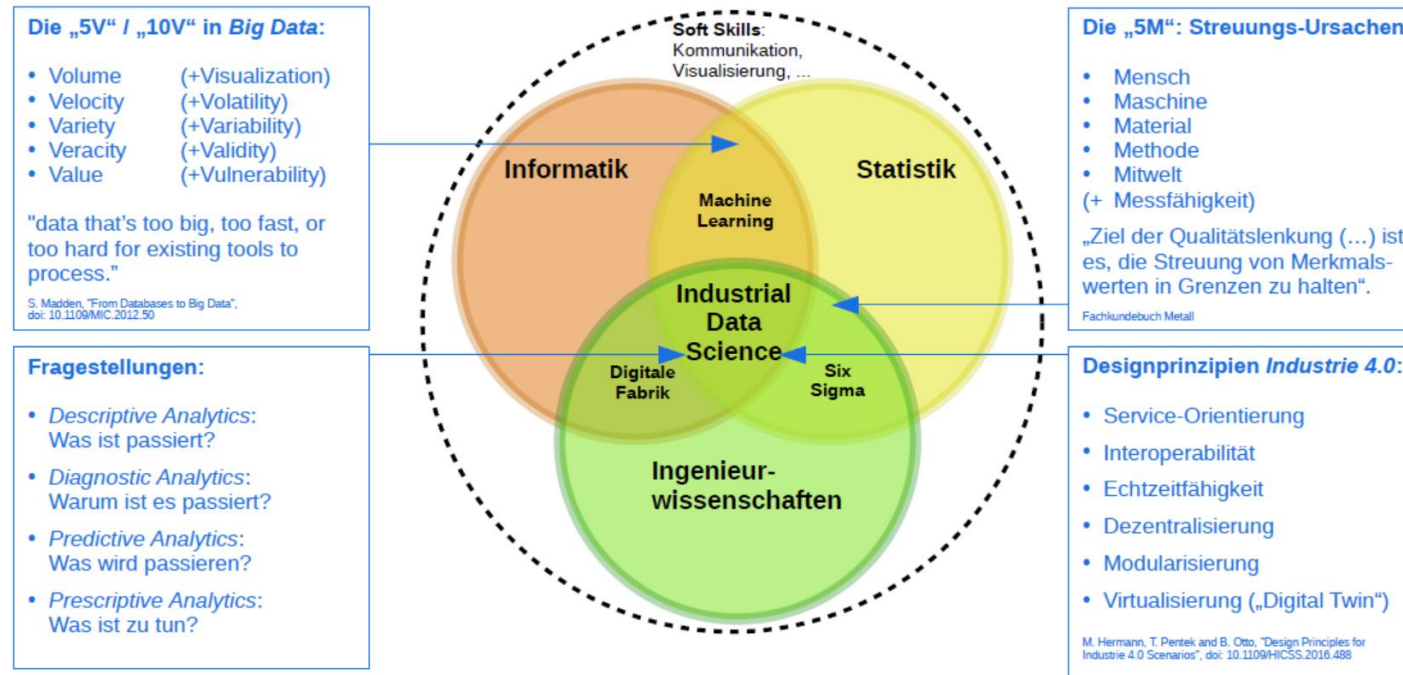
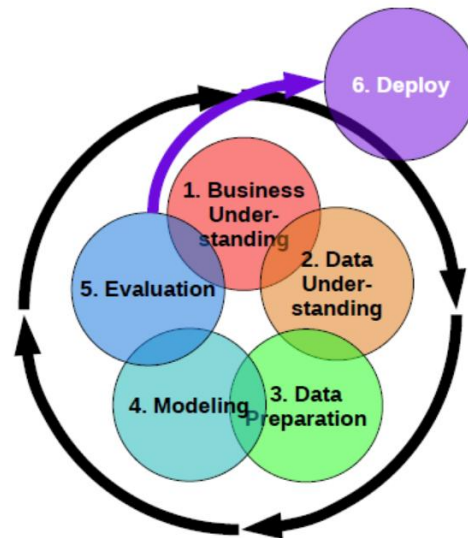


Abb. 3.5 Verzahnung virtueller Entwicklungs- oder Planungsschritte mit der realen Umsetzung (Quelle: Volkswagen AG)

U. Bracht, D. Geckler, S. Wenzel:
Digitale Fabrik: Methoden und Praxisbeispiele,
2. Auflage, Springer 2018, S. 74



3.2 Data Science Projektmanagement mit CRISP-DM

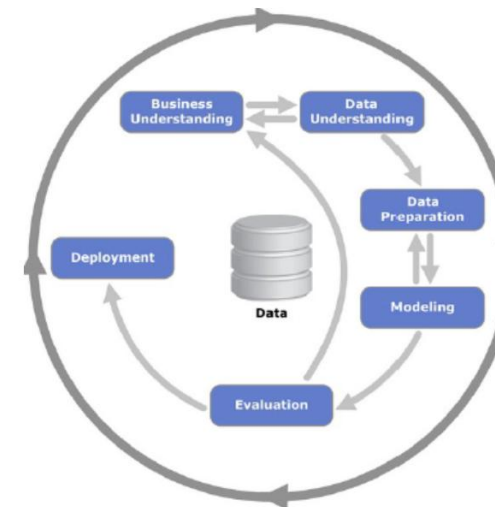


CRISP-DM

(Cross Industry Standard
Process for Data Mining)

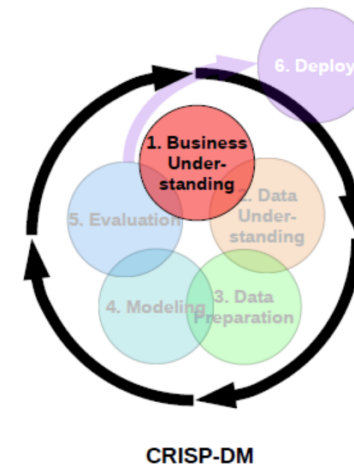
In offiziellen Dokumenten bitte
die Original-Darstellung verwenden:

https://upload.wikimedia.org/wikipedia/commons/b/b9/CRISP-DM_Process_Diagram.png



Inhalt:

- 1) Verständnis des Problems
- 2) Vereinbarung von Zielen
- 3) Vereinbarung von Metriken



Beispiel:

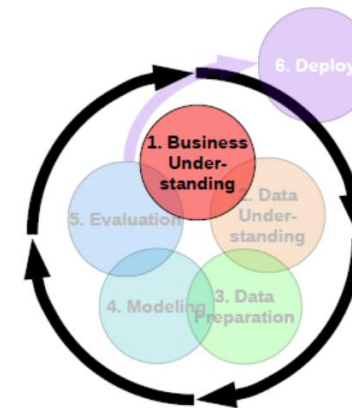
Ziel: Erkennung eines fehlerhaften Bauteils
(Klassifikationsproblem)

Eine geeignete Metrik kann auf Basis der
Confusion Matrix definiert werden:

	Tatsächliche Klasse	
	Positive	Negative
Vorhergesagte Klasse	Positive	True Positive 0
	Negative	False Negative 1
		Negative
Positive	False Positive 0	True Positive 99
Negative	True Negative 1	False Negative 0

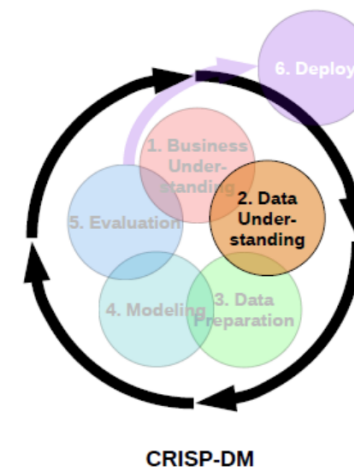
Anzahl
„Als Fehlerhaft deklariert“

Accuracy: $(TP+TN) / [alle] = 99/100 = 99\%$
Spezifität: $TN / [tatsächlich\ neg.] = 99/99 = 100\%$
Sensitivität: $TP / [tatsächlich\ pos.] = 0/1 = 0\%$



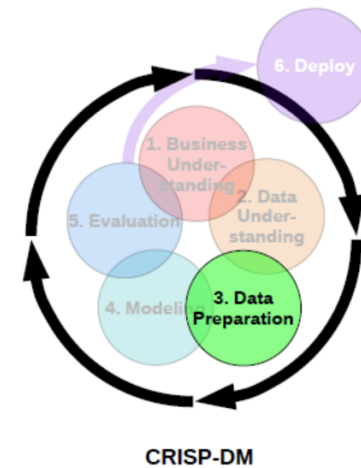
Inhalt:

- 1) Erschließung der Datenquellen
- 2) Übersicht über Datenqualität und -vollständigkeit
- 3) Visualisierung



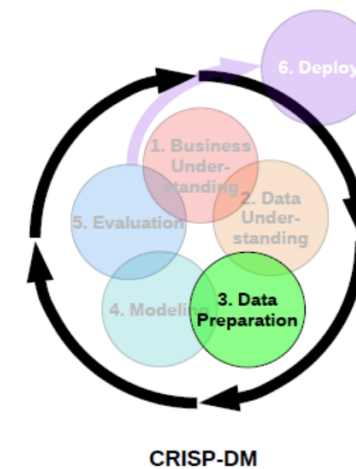
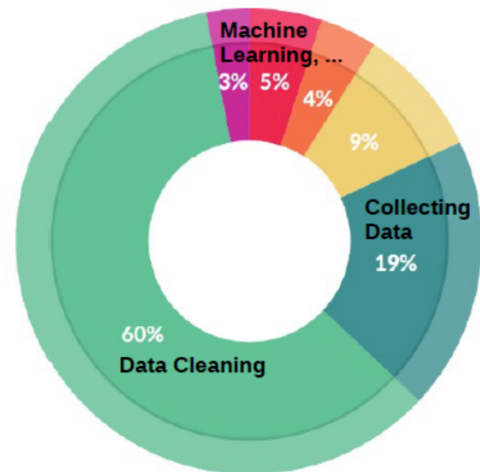
Inhalt:

- 1) Datenreinigung („Cleansing“)
- 2) Strategie für fehlende Daten entwickeln (Drop / Data Imputation)
- 3) Feature Engineering



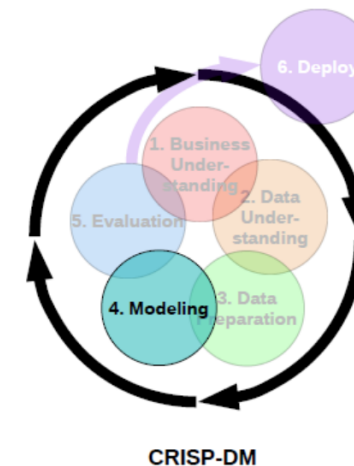
Nach: https://visit.figure-eight.com/rs/416-ZBE-142/images/CrowdFlower_DataScienceReport_2016.pdf

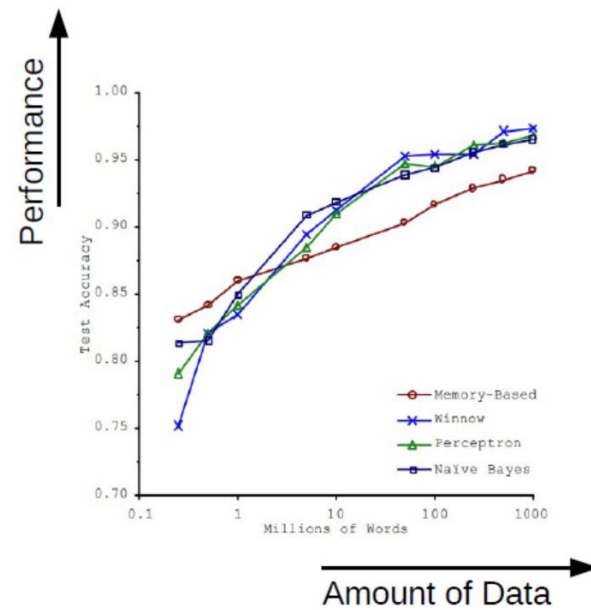
What data scientists spend the most time doing



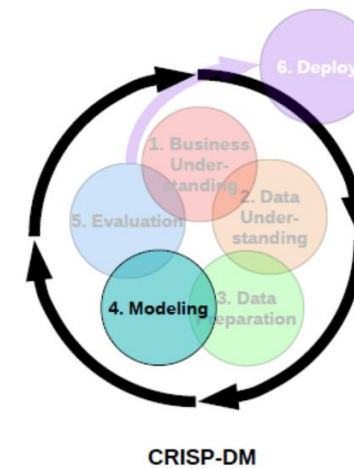
Inhalt:

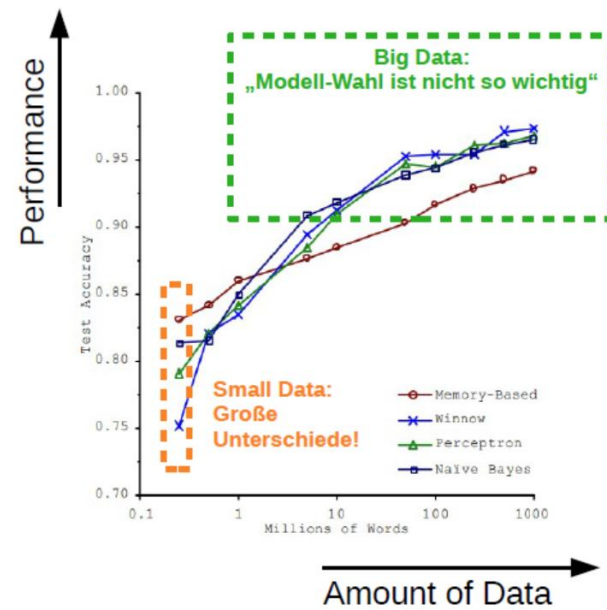
- 1) Anwendung geeigneter Algorithmen
und/oder Machine Learning-Verfahren
- 2) Training (Trainings-Datensatz),
Evaluierung (unabhängiger Evaluations-Datensatz)



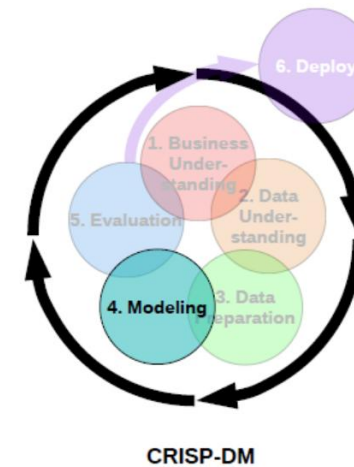


M. Banko, E. Brill: Scaling to very very large corpora for natural language disambiguation,
ACL '01: Proceedings of the 39th Annual Meeting on Association for Computational Linguistics,
July 2001, Pages 26–33



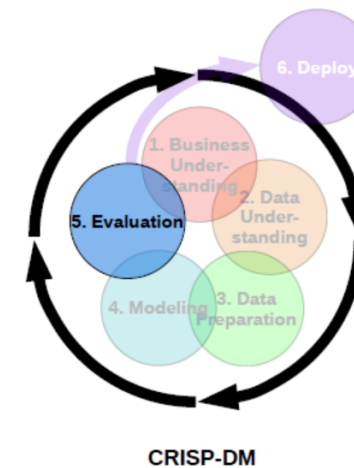


M. Banko, E. Brill: Scaling to very very large corpora for natural language disambiguation, ACL '01: Proceedings of the 39th Annual Meeting on Association for Computational Linguistics, July 2001, Pages 26–33



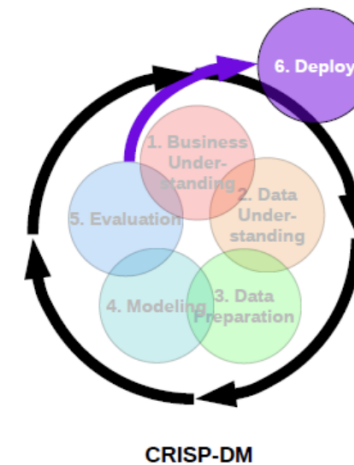
Inhalt:

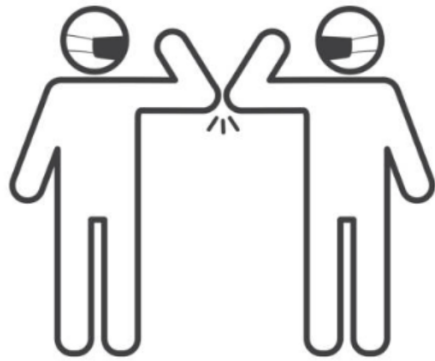
- 1) Verifizierung eines finalen Modells:
Können vereinbarte (ML-)Metriken auf einem weiteren unabhängigen Test-Set erreicht werden?
- 2) Validierung:
Wird der erwartete Kundennutzen generiert?



Inhalt:

- 1) Übergabe an den Kunden
- 2) Integration in die Systeme des Kunden





1. Die Elbow-Methode im Clustering
2. Die Elbow-Methode bei der PCA
3. Data Science Projektmanagement

**Abschließend: Zusammenfassung
und Fragen**





Entstehung von Knochenentwicklung bei Kindern, <https://www.bonepart.com>

1. Einführung und Überblick

2. Abgrenzung Data Science und Data Analytics

3. Zusammenfassung und Semesterplanung

Umfrage: Was erwarten Sie von der Vorlesung?
<https://www.menti.com/aljhvyxrh85n>
(oder <https://www.menti.com> -> Voting Code 5359 5510)

(VL01) Besonderer Fokus auf folgende Inhalte:

- Python Basics
- Wie lese ich eine Datei ein?
- Wie speichere ich den Inhalt in einer Liste mit Strings, die jeweils eine Zeile beinhalten?
- .csv Datei in pandas einlesen und Spalte hinzufügen mit einfachem Feature-Engineering (z.B. Summe zweier anderer Spalten)
- einfache Visualisierung erstellen (Scatterplot, Linienplot, ...)



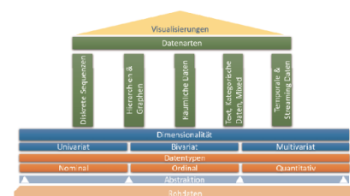
1. Grundbegriffe Data Science / Data Analytics

2. Python-Grundlagen

3. Einfache Visualisierungen

(VL02) Besonderer Fokus auf folgende Inhalte:

- Data Science Venn Diagramm
- Unterschiede Syntax/Semantik, Strukturierte/Unstrukturierte Daten, Supervised/Unsupervised Learning
- Welche "Analytics" gibt es im Gartner Ascendancy Model?
- Python Basics (Zahlentypen, Boolesche Operationen+Indizierung, String-Manipulationen, Type casting, Listen/Tuple/Dictionary+Comprehension, Slicing, Kontrollstrukturen)
- Visualisierungsmöglichkeiten eines numerischen Vektors (Histogramm, KDE, Violinplot, Stripplot, Boxplot inkl. Verständnis der definierenden Elemente -- auch Quantile, vgl. spätere VL)

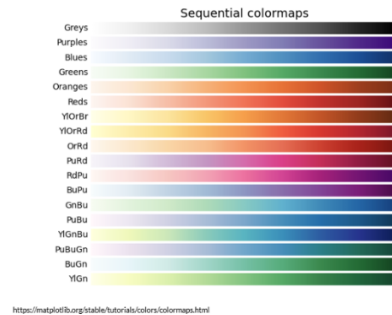


1. Visualisierung und Encodings

2. DecisionTreeClassifier

(VL03) Besonderer Fokus auf folgende Inhalte:

- Matplotlib: Figure erstellen, Scatter- oder Linienplot erstellen basierend auf gegebene x- und y-Vektoren; Achsenbeschriftung und Titel setzen
- Verständnis der Werteskalen Kategorial (Ordinal, Nominal) vs Kardinal
- Ordinal Encoding vs. One Hot Encoding (für welche Werteskalen anwendbar?)
- Konfusionsmatrix und deren Bestandteile

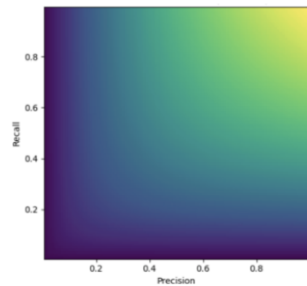


1. Die Holdout-Methode

2. Indizierung in Pandas

(VL04) Besonderer Fokus auf folgende Inhalte:

- Grundsätzliche Funktionsweise eines Entscheidungsbaums für eine Klassifikation
- Decision Tree Algorithmus zum Training und Gini Impurity
- Wie funktioniert Inference (.predict) beim Entscheidungsbaum?
- Holdoutmethode und Train/Validierungs/Test-Split; Diskussion von Overfit; Hyperparametertuning und die Rolle des Validierungsdatensatzes
- Wahrscheinlichkeitsrechnung seltener Ereignisse (Beispiel Rot-Grün-Schwäche)
- sklearn Grundprinzipien (.fit, .predict)
- Slicing mit .loc/.iloc in pandas



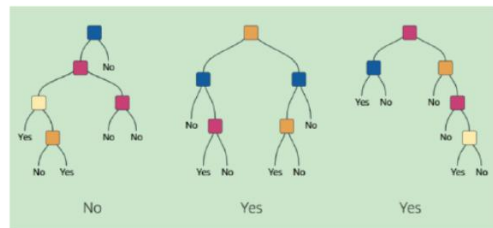
1. Objektorientierung in Python

2. Klassifikationsmetriken

3. Regression im Entscheidungsbaum

(VL05) Besonderer Fokus auf folgende Inhalte:

- Bei einer Klassifikation: Scores vs Predictions
- Klassifikationsmetriken Precision, Recall, Sensitivität, Spezifität, Accuracy, F1-Score
- Rolle des Thresholds und Auswirkung auf Klassifikationsmetriken (Precision-Recall-Tradeoff)
- Entscheidungsbaumalgorithmus für eine Regression: Wie funktioniert .train, wie funktioniert .predict?
- Wie viele unterschiedliche Werte kann ein Regressionsbaum der Tiefe k vorhersagen?



1. Regressionsmetriken

2. Random Forest

(VL06) Besonderer Fokus auf folgende Inhalte:

- Metriken RMSE/MSE und R^2 bei einer Regression
- Random Forest: Grundidee, Ziehen mit Zurücklegen
- Mittelung/Majority Vote über die Predictions sowie Ausnutzung der Standardabweichung zur Unsicherheitsmessung
- Was ist eine Validierungskurve? Wird diese auf Trainings-, Validierungs- oder Testdaten erhoben?

A | B | C
12
13
14

1. Lagemaße

2. Informationsvisualisierung

3. Streumaße

(VL07) Besonderer Fokus auf folgende Inhalte:

- Lagemaße: Quantile, Median, arithmetischer Mittelwert sowie Diskussion von Outliern
- Funktionsweise eines Pair Plots
- Gestalt Laws, Kontextsensitivität
- Streumaße: Varianz, Standardabweichung, IQR, sowie Diskussion von Outliern
- grundsätzliche Bedeutung des Gesetz der großen Zahlen



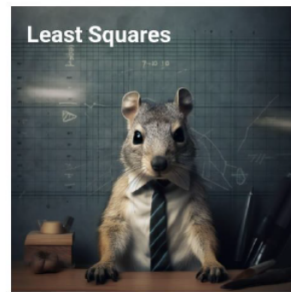
1. Numpy

2. Best practices der Informationsvisualisierung

3. Korrelation

(VL08) Besonderer Fokus auf folgende Inhalte:

- Numpy Basics
- Pearson-Korrelation
- Was können Ränge in einem numerischen Vektor definiert werden?
- Spearman-Korrelation
- Diskussion von Outliern



von: Jason Taylor (Dabuku)
(generiert mit Midjourney)

Least Squares is like a tailor adjusting a suit to fit a customer perfectly. Imagine a customer trying on a suit that doesn't fit quite right - it's too loose in some places and too tight in others. The tailor measures the differences between the suit and the customer's body, then makes adjustments to minimize these discrepancies. By finding the smallest sum of these squared differences, the tailor ensures the suit fits as closely as possible to the customer's measurements. Least Squares algorithm works similarly, adjusting the parameters of a mathematical model to minimize the sum of the squared differences between the predicted values and the actual data, resulting in the best possible fit.

1. Lineare Regression

2. Gesundheitsdaten

3. Gradient Descent

(VL09) Besonderer Fokus auf folgende Inhalte:

- Methode der kleinsten Quadrate
- Zusammenhang Steigungskoeffizient einer lineare Regression zur Pearsonkorrelation
- Zusammenhang $R^2 = \text{cor}(x,y)^2$ (s. Übung)
- Exakte Berechnung der Koeffizienten: Wie würde man vorgehen?
- Gradient Descent Verfahren (Algorithmus!)
- Was versteht man unter Lead Time Bias? Welche Kritik kann man an der "5 years survival rate" anbringen?

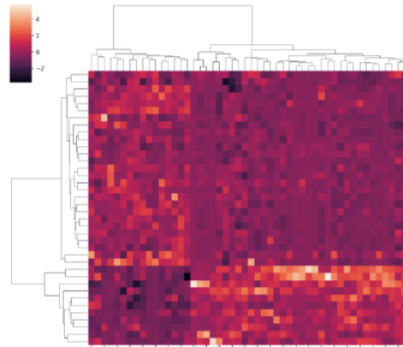


1. Dimensionsreduktion

2. Studiendesign

(VL10) Besonderer Fokus auf folgende Inhalte:

- PCA: Grundideen und typische Anwendungen
- t-SNE: Grundideen und typische Anwendungen
- Unterscheidung Prävalenz/Inzidenz und Phasen von Zulassungsstudien



1. Wiederholung

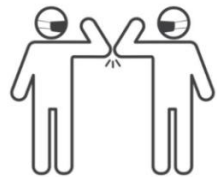
2. Personalisierte Medizin

3. Clusterverfahren

(VL11) Besonderer Fokus auf folgende Inhalte:

- Cluster Map (Kombination aus Heat Map und Dendrogrammen)
- Hierarchisches Clustering und Dendrogramm
- Zusammenhang Punktmatrix und Clustermetrik
- Verschiedene Linkage-Methoden (insb. single / complete / average)

Wir haben in dieser Vorlesung auch über *Feature Importance* gesprochen, auch am Beispiel einer Linearen Regression. Hier wurde die Notwendigkeit der *Standardisierung* der Daten für eine Interpretation der Koeffizienten angesprochen.



1. Die Elbow-Methode im Clustering

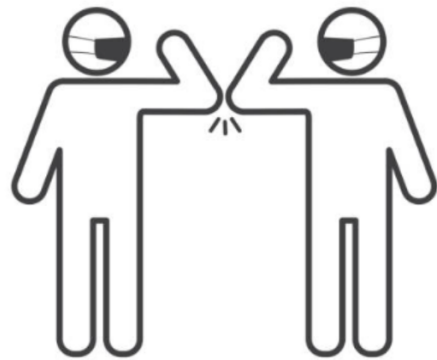
2. Die Elbow-Methode bei der PCA

3. Data Science Projektmanagement

Abschließend: Zusammenfassung
und Fragen

(VL12) Besonderer Fokus auf folgende Inhalte:

- Explained Variance Ratio bei der PCA
- Elbow-Methode zur Bestimmung eines geeigneten Hyperparameters (allgemein und konkret bei der Suche nach einer passenden PCA-Dimension oder einer passenden Cluster-Anzahl bei hierarchischem oder kmeans-Clustering)
- Inertia zur Clusterbewertung zur Bestimmung eines geeigneten $n_clusters$ -Parameters
- CRISP-DM



1. Die Elbow-Methode im Clustering
2. Die Elbow-Methode bei der PCA
3. Data Science Projektmanagement

**Abschließend: Zusammenfassung
und Fragen**

Vielen Dank für Ihre Aufmerksamkeit!